

Library Management System using SQL Server

Project Overview

Project Title: Library Management System

Level: Intermediate

Database: SQL Server (`sql_project2`)

This project demonstrates the implementation of a Library Management System using Microsoft SQL Server. The project covers database design, relationship management, CRUD operations, business reporting, stored procedures, and advanced SQL queries.

Unlike a tutorial-based implementation, the majority of the queries, relationships, and business logic were developed independently. The project involved resolving foreign key conflicts, fixing datatype mismatches, enforcing referential integrity, and converting PostgreSQL-based solutions into SQL Server syntax.

Objectives

- Design a relational database for a library management system.
 - Implement Primary Key and Foreign Key constraints.
 - Maintain referential integrity between tables.
 - Perform CRUD operations.
 - Create business reports using SQL.
 - Develop stored procedures for automation.
 - Analyze library transactions and member activity.
 - Create reporting tables using SQL Server.
-

Database Schema

The project consists of the following tables:

Branch

Stores branch information and branch managers.

Employees

Stores employee information and branch assignments.

Members

Stores member registration information.

Books

Stores book details, categories, rental prices, and availability status.

Issued_Status

Tracks all book issue transactions.

Return_Status

Tracks all returned books.

Database Relationships

The following relationships were implemented using Foreign Keys:

```
ALTER TABLE issued_status
ADD FOREIGN KEY (issued_member_id)
REFERENCES members(member_id);

ALTER TABLE issued_status
ADD CONSTRAINT fk_books
FOREIGN KEY (issued_book_isbn)
REFERENCES books(isbn);

ALTER TABLE issued_status
ADD CONSTRAINT fk_employees
FOREIGN KEY (issued_emp_id)
REFERENCES employees(emp_id);

ALTER TABLE employees
ADD CONSTRAINT fk_employee
FOREIGN KEY(branch_id)
REFERENCES branch(branch_id);

ALTER TABLE return_status
ADD CONSTRAINT fk_return
FOREIGN KEY (issued_id)
REFERENCES issued_status(issued_id);
```

SQL Concepts Used

Joins

- INNER JOIN
- LEFT JOIN
- SELF JOIN

Aggregations

- COUNT()
- SUM()
- AVG()

Filtering

- WHERE
- HAVING

Window Functions

- ROW_NUMBER()

Common Table Expressions (CTEs)

Subqueries

- NOT EXISTS
- IN

Stored Procedures

Date Functions

- GETDATE()
- DATEADD()
- DATEDIFF()

SQL Server Reporting

- SELECT INTO
 - GROUP BY
 - ORDER BY
-

CRUD Operations

Task 1: Create a New Book Record

```
INSERT INTO books
VALUES
(
  '978-1-60129-45-2',
  'To kill a mockingbird',
  'Classic',
  1,
  'Harper Lee',
  'J.B Lippincott & Co',
  6.00
);
```

Task 2: Update Existing Member Address

```
UPDATE members
SET member_address='420 GdK TG'
WHERE member_id='C101';
```

Task 3: Delete an Issued Record

```
DELETE FROM issued_status
WHERE issued_id='IS121';
```

Business Queries & Analysis

Task 4: Retrieve Books Issued by a Specific Employee

```
SELECT
  i.issued_book_name,
  e.emp_id
FROM issued_status i
INNER JOIN employees e
```

```
ON i.issued_emp_id=e.emp_id
WHERE i.issued_emp_id='E101';
```

Task 5: Members Who Issued More Than One Book

```
SELECT
    i.issued_member_id,
    COUNT(*) AS number
FROM issued_status i
INNER JOIN members m
    ON i.issued_member_id=m.member_id
GROUP BY i.issued_member_id
HAVING COUNT(*)>=2
ORDER BY number DESC;
```

Task 6: Category Analysis

```
SELECT
    category,
    COUNT(*) AS no_of_books
FROM books
GROUP BY category
ORDER BY no_of_books DESC;
```

Task 7: Total Rental Income by Category

```
SELECT
    b.category,
    SUM(b.rental_price) AS total_income,
    COUNT(i.issued_book_isbn) AS books_issued
FROM books b
INNER JOIN issued_status i
    ON b.isbn=i.issued_book_isbn
GROUP BY b.category
ORDER BY total_income DESC;
```

Task 8: Member Registration Analysis

Members Registered in 2024

```
SELECT *  
FROM members  
WHERE YEAR(reg_date)=2024;
```

Members Registered in Last 2 Years

```
SELECT *  
FROM members  
WHERE reg_date>=DATEADD(YEAR,-2,GETDATE());
```

Task 9: Employee, Branch & Manager Report

```
SELECT  
    b.*,  
    e.emp_id,  
    e.emp_name,  
    m.emp_name AS manager_name  
FROM employees e  
INNER JOIN branch b  
    ON e.branch_id=b.branch_id  
INNER JOIN employees m  
    ON b.manager_id=m.emp_id;
```

Reporting Tables

Task 10: Create Expensive Books Table

```
SELECT *  
INTO expensive_books  
FROM books  
WHERE rental_price>=6.00;
```

Task 11: Books Not Yet Returned

```
SELECT *
FROM issued_status i
WHERE NOT EXISTS
(
    SELECT r.issued_id
    FROM return_status r
    WHERE r.issued_id=i.issued_id
);
```

Task 12: Overdue Book Analysis

```
SELECT
    m.member_id,
    m.member_name,
    i.issued_book_name,
    i.issued_date,
    DATEDIFF(DAY,i.issued_date,r.return_date)-30 AS days_overdue
FROM members m
INNER JOIN issued_status i
    ON m.member_id=i.issued_member_id
INNER JOIN return_status r
    ON i.issued_id=r.issued_id
WHERE DATEDIFF(DAY,i.issued_date,r.return_date)>30;
```

Stored Procedures

Task 13: Update Book Status on Return

```
CREATE PROCEDURE get_updated
AS
BEGIN
    UPDATE books
    SET status=1
    WHERE isbn IN
    (
        SELECT return_book_isbn
        FROM return_status
    )
```

```
);  
END;
```

```
EXEC get_updated;
```

Branch Performance Reporting

Task 14: Branch Performance Report

```
SELECT *  
INTO branch_report_performance  
FROM  
(  
    SELECT  
        e.branch_id,  
        COUNT(i.issued_emp_id) AS no_of_issuedbooks,  
        COUNT(r.return_id) AS no_of_returned_books,  
        SUM(b.rental_price) AS revenue  
    FROM employees e  
    INNER JOIN issued_status i  
        ON e.emp_id=i.issued_emp_id  
    LEFT JOIN return_status r  
        ON i.issued_id=r.issued_id  
    INNER JOIN books b  
        ON i.issued_book_isbn=b.isbn  
    GROUP BY e.branch_id  
) report;
```

Active Members Analysis

Task 15: Active Members Table

```
SELECT *  
INTO Active_members  
FROM  
(  
    SELECT DISTINCT issued_member_id  
    FROM issued_status
```



```
WHERE DATEDIFF(MONTH, issued_date, GETDATE()) <= 2
) members;
```

Employee Performance Analysis

Task 16: Top 3 Employees by Books Processed

```
SELECT TOP 3
    e.emp_id,
    e.emp_name,
    COUNT(i.issued_id) AS no_of_processed,
    e.branch_id
FROM employees e
INNER JOIN issued_status i
    ON e.emp_id=i.issued_emp_id
GROUP BY
    e.emp_id,
    e.emp_name,
    e.branch_id
ORDER BY no_of_processed DESC;
```

High Risk Book Analysis

Task 17: Damaged Book Investigation

```
SELECT
    m.member_name,
    b.book_title,
    COUNT(*) AS no_of_damaged_booksissued
FROM members m
INNER JOIN issued_status i
    ON m.member_id=i.issued_member_id
INNER JOIN books b
    ON i.issued_book_isbn=b.isbn
WHERE b.status='damaged'
GROUP BY
    m.member_name,
    b.book_title
```

```
HAVING COUNT(*)>=2  
ORDER BY no_of_damaged_booksissued DESC;
```

Parameterized Stored Procedure

Task 18: Book Issue Management Procedure

```
ALTER PROCEDURE get_data_new  
    @book_id NVARCHAR(100)  
AS  
BEGIN  
    DECLARE @status BIT;  
  
    SELECT @status=status  
    FROM books  
    WHERE isbn=@book_id;  
  
    IF @status=1  
    BEGIN  
        UPDATE books  
        SET status=0  
        WHERE isbn=@book_id;  
  
        PRINT 'Book is issued successfully';  
    END  
    ELSE  
    BEGIN  
        PRINT 'Book is not available';  
    END  
END;
```

Execute Procedure

```
EXEC get_data_new '978-0-14-143951-8';
```

Overdue Fine Calculation

Task 19: Fine Calculation for Returned Books

```
SELECT
    i.issued_member_id,
    COUNT(*) AS over_due_books,
    SUM((DATEDIFF(DAY,i.issued_date,r.return_date)-30)*0.5) AS over_due_amount
FROM issued_status i
LEFT JOIN return_status r
    ON i.issued_id=r.issued_id
WHERE DATEDIFF(DAY,i.issued_date,r.return_date)>30
GROUP BY i.issued_member_id;
```

Task 20: Fine Calculation for Books Not Yet Returned

```
SELECT
    issued_member_id,
    issued_book_name,
    issued_date,
    (DATEDIFF(DAY,issued_date,GETDATE())-30)*0.5 AS fine_amount
FROM issued_status i
WHERE i.issued_id NOT IN
(
    SELECT issued_id
    FROM return_status
)
AND DATEDIFF(DAY,issued_date,GETDATE())>30;
```

This additional analysis calculates fines dynamically for books that are still with members and have not yet been returned.

Key Learning Outcomes

Through this project, I gained hands-on experience in:

- Relational Database Design
- Primary Key & Foreign Key Implementation
- Referential Integrity Management
- SQL Server Stored Procedures

- Business Reporting
 - Query Optimization
 - Data Cleaning
 - SQL Server Date Functions
 - CTEs and Window Functions
 - Real-world Database Problem Solving
-

Technologies Used

- Microsoft SQL Server
 - SQL Server Management Studio (SSMS)
-

Author

Kruthik Teliseeri

Aspiring Data Analyst | SQL Server | Power BI | Python | Machine Learning

This project was completed independently to strengthen SQL Server, database design, reporting, and business-oriented SQL problem-solving skills.